DNN Programming Assignment 2

RNN, LSTM, GRU and Transformer

RAKSHANTHA M · Student ID 2026AK05368

Variant: ID ends in 8 (even), so 5(c) = RNN clipping · hidden size 64, d = 128, seeds 0, 1, 2

1 Problem 1 — RNN, LSTM and GRU

1.1 Setup

- **Data.** UCI HAR: 7,352 training / 2,947 test windows, 128 steps × 9 channels, 6 near-balanced classes (Figure 1a); training and test subjects do not overlap.
- **Split and normalisation.** Per seed, 4 of the 21 training subjects are held out for validation (seed 0: 8, 17, 21, 25 · seed 1: 15, 16, 25, 29 · seed 2: 5, 7, 11, 25). Per-channel mean and std come from the other 17 subjects only.
- **Adding problem.** Provided generator, 10,000 / 1,000 / 1,000 sequences (data seeds 0 / 1 / 2), with an assert test for two markers (one per half) and target = sum of marked values.
- **Training.** Hidden size 64, Adam (lr 1e-3), batch 64, 30 epochs, seeds 0–2 (mean ± sample std). HAR uses the best-validation checkpoint; the adding problem clips gradients at 1.0. Colab T4, PyTorch 2.14.1.
- **Assumption.** The variants table was not in the released brief, so hidden size 64, d = 128 and seeds 0–2 were used.

1.2 Verification of the recurrent cells

Library weights are copied into my cells, which keep one bias b = b_ih + b_hh per gate block (the GRU's b_hn, inside r ⊙ (W_hn h + b_hn), is set to zero before comparing — assumption). All three cells match within $\mathbf{1.5 \times 10^{-7}}$. Each gate block has h·d + h·h + h = **4,736** parameters, so totals are k × 4,736; PyTorch keeps a second bias per block and reports k × 64 more (Table 1).

Table 1. Cells from scratch vs PyTorch (d = 9, h = 64, tolerance 1e-5).

Cell	Max |diff|	Hand count k·(hd+hh+h)	My code	PyTorch	Difference (k·h)
RNN (k = 1)	1.49×10^{-7}	4,736	4,736	4,800	64
LSTM (k = 4)	5.96×10^{-8}	18,944	18,944	19,200	256
GRU (k = 3)	8.94×10^{-8}	14,208	14,208	14,400	192

1.3 HAR classification

- GRU **0.900 ± 0.007** and LSTM **0.897 ± 0.008**; RNN **0.810 ± 0.011** (Table 2): the plain RNN is lower because its gradient vanishes over 128 steps (Figure 3). Per-epoch time is 0.30–0.32 s for all three.
- Best model (GRU, seed 0, 0.908): the main error is **sitting → standing (98 windows)** (Figure 1b).

Table 2. HAR test results (3 seeds, same budget for all models).

Model	Test accuracy	Macro-F1	Parameters	s / epoch
RNN	0.8104 ± 0.0107	0.8058 ± 0.0097	5,190	0.30
LSTM	0.8974 ± 0.0083	0.8977 ± 0.0090	19,590	0.31
GRU	**0.9001 ± 0.0066**	**0.8998 ± 0.0066**	14,790	0.32

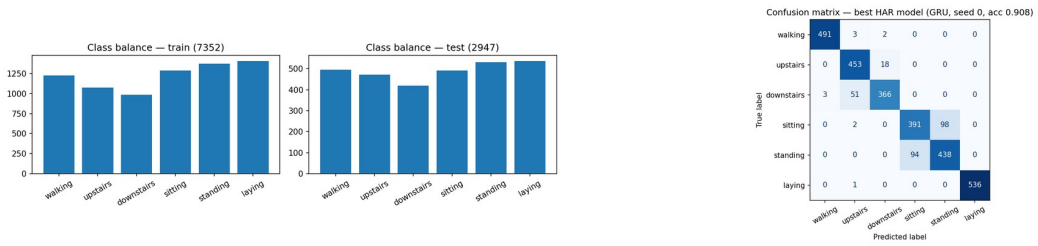


Figure 1. (a) Class balance of the HAR training and test sets. (b) Confusion matrix of the best HAR model (GRU, seed 0).

1.4 Adding problem

- **RNN** solves 1/3 at T = 50, then stays at the baseline. **LSTM** solves 3/3 at T = 50 and 2/3 at T = 100, then none. **GRU** solves every run at every length (Table 3).

- At T = 200 the GRU's loss sits at the baseline for the first 8 epochs, then drops sharply (Figure 2a).

Table 3. Test MSE (mean ± std, 3 seeds); solved runs (MSE < 0.05) in brackets; baseline = always predict 1.0.

Model	T = 50	T = 100	T = 200	T = 400
RNN	0.0817 ± 0.0726 (1/3)	0.1698 ± 0.0003 (0/3)	0.1685 ± 0.0006 (0/3)	0.1543 ± 0.0010 (0/3)
LSTM	0.0077 ± 0.0076 (3/3)	0.0632 ± 0.0913 (2/3)	0.1677 ± 0.0002 (0/3)	0.1531 ± 0.0003 (0/3)
GRU	**0.0002 ± 0.0000 (3/3)**	**0.0006 ± 0.0002 (3/3)**	**0.0006 ± 0.0002 (3/3)**	**0.0027 ± 0.0025 (3/3)**
Baseline	0.1754	0.1697	0.1679	0.1527

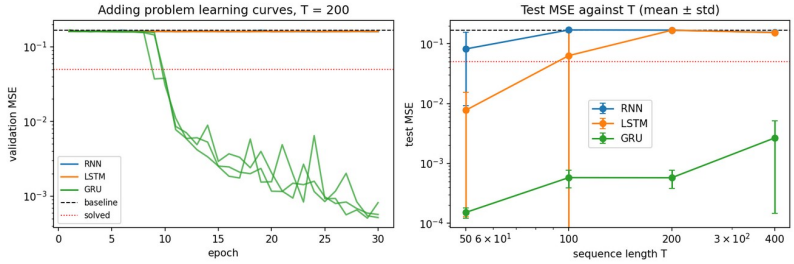


Figure 2. (a) Validation MSE per epoch at T = 200, all seeds. (b) Test MSE against T (log scales).

1.5 Gradients and ablations

5(a) Gradient norms. Before training, ‖∂L/∂h_t‖ falls by about 20 orders of magnitude from t = 99 to t = 0 in all three models (to exactly 0 in float32 for the RNN and LSTM). After training the RNN still falls from 1.2 × 10^{-3} to **7.3 × 10^{-13}**, while the LSTM stays near **4 × 10^{-5}** and the GRU near **1.5 × 10^{-4}** (Figure 3).

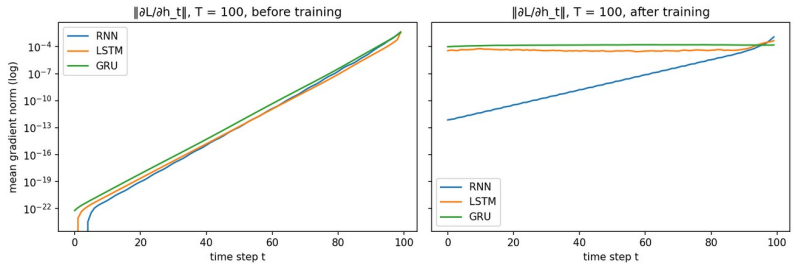


Figure 3. Mean ‖∂L/∂h_t‖ against time step t at T = 100, before (left) and after (right) training.

5(b) Hidden-size sweep. The GRU stays at 0.891–0.900 for every size (1,398 to 54,150 parameters); the LSTM drops to 0.816 ± 0.046 at h = 16 and peaks at 0.906 at h = 128 (71,942 parameters) (Figure 4).

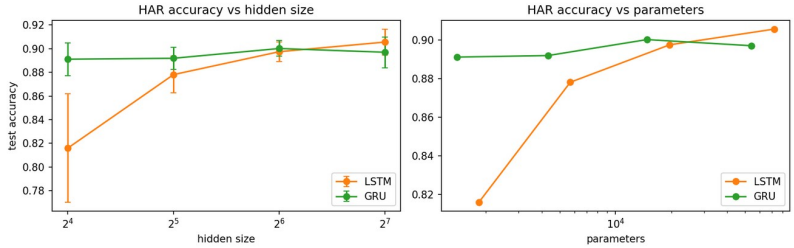


Figure 4. HAR test accuracy (mean ± std) against hidden size (left) and parameter count (right), LSTM and GRU.

5(c) Clipping, vanilla RNN, T = 200. Clip = 1.0 and no clipping give the same result (Table 4): **0.1685 ± 0.0006**, 0/3 solved, and the clipped runs reproduce the main runs exactly. The pre-clip gradient norm averaged 0.30 and exceeded 1.0 on only about 1.5% of steps (Table 4, Figure 5).

Table 4. Ablation 5(c), vanilla RNN, T = 200 (3 seeds).

Setting	Test MSE	Solved	Mean pre-clip grad. norm	Steps with norm > 1.0
Clip = 1.0	0.1685 ± 0.0006	0/3	0.30	about 1.5%
No clipping	0.1685 ± 0.0006	0/3	0.31	about 1.5%

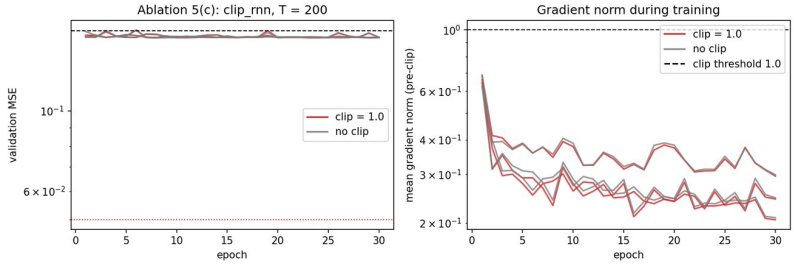


Figure 5. Ablation 5(c): (a) validation MSE and (b) mean pre-clip gradient norm per epoch, with the 1.0 threshold.

1.6 Task 5(d) — Analysis: memory, failure with T, trade-offs, wearable choice

Memory (Figure 3). Before training, all three models show the same steep decline of $\|\partial L/\partial h_t\|$ from t = 99 to t = 0 (about 20 orders of magnitude), because every backward step shrinks the signal when the weights are random. After training the RNN's gradient at t = 0 is still about 10^9 times smaller than at t = 99 (ratio 6×10^{-10}), so the first marker cannot teach it anything — the vanishing-gradient problem. The LSTM keeps about 8% (ratio 0.076) and the GRU about 64% (ratio 0.64), because their gates give additive paths (the LSTM cell state, the GRU update-gate interpolation) along which gradients flow back without repeatedly shrinking. This matches Table 3: the longer the gradient survives, the longer the sequences solved. Caution: the curves are for seed 0 at T = 100, where the trained RNN had stalled at the baseline, so its curve describes a model that never learned.

Failure as T grows (Table 3, Figure 2). An MSE near the baseline means the model predicts the average target 1.0 instead of adding the marked values. The RNN is unreliable already at T = 50 (1/3 solved) and fails at every longer length. The LSTM solves 3/3 at T = 50, 2/3 at T = 100 (the third stays at 0.1685) and none from T = 200. The GRU solves every run at every length; its error grows slowly, from 0.0002 at T = 50 to 0.0027 at T = 400 (± 0.0025). The baseline depends on the test set (0.1527 at T = 400), so the RNN and LSTM scores of 0.154 and 0.153 there are still baseline-level. In Figure 2a the GRU sits at the baseline for 8 epochs, then drops sharply, so the LSTM might also succeed with more epochs or a forget-gate bias of 1 (nn.LSTM uses 0); I did not test this. Clipping changed nothing in 5(c), as only about 1.5% of steps exceeded norm 1.0 (Table 4): the problem is vanishing, not exploding, gradients.

Accuracy, parameters and time (Table 2, Figure 4). The GRU and LSTM are indistinguishable at h = 64 (gap 0.003, below either standard deviation), but the GRU has about 25% fewer parameters (14,790 vs 19,590). The RNN has the fewest parameters (5,190) but is about 9 points less accurate and no faster: seconds per epoch are almost identical for all three (0.30–0.32 s), which suggests the 128 sequential steps, not the parameter count, dominate the time. In the sweep (Figure 4) the GRU is nearly flat, from 0.891 at h = 16 (1,398 parameters) to 0.900 at h = 64, whereas the LSTM needs capacity: 0.816 ± 0.046, 0.878, 0.897 and 0.906 at h = 16, 32, 64 and 128. The last 0.8-point gain costs 3.7 times the parameters, raises the time per epoch to 0.41 s (others 0.29–0.31 s) and is within one standard deviation of the GRU at h = 64. The remaining errors are mainly sitting vs standing (F1 0.80 and 0.82 against 0.91–1.00 for the other classes), two static postures with similar signals, which more capacity does not fix.

Wearable choice. I would use a GRU with hidden size 16: 0.891 ± 0.014 accuracy with 1,398 parameters (about 5.6 KB in float32), within one standard deviation of the LSTM at h = 64 (14 times the parameters), while the RNN (0.810) and the LSTM at h = 16 (0.816) lose about 8 points. The GRU was also the only cell to solve the adding problem at every length (tested at h = 64, not 16). The cost is a larger seed spread (± 0.014 vs ± 0.007), so if memory is not tight, h = 64 is safer.

Limits. All comparisons use one fixed budget (30 epochs, lr 1e-3, no tuning) and three seeds, so gaps of 0.003–0.006 are not evidence of a difference, and the RNN and LSTM failures describe this budget, not the architectures in general. Seconds per epoch on a T4 say nothing about battery use; parameter count is only a proxy, so energy per inference should be measured on the device.

2 Problem 2 — Transformer from scratch

2.1 Attention tests and hand-checked example

- **Unit tests pass** (Table 5): the attention weights, the scaled dot-product output and the multi-head layer all match the references to about 10^{-7}, and the causal mask is exact.
- **Hand example** (own, as the instructor's was not supplied): Q = K = [[1, 0], [0, 1], [1, 1]], V = [[1, 2], [3, 4], [5, 6]]. Row 1: scores [0.707, 0, 0.707] → weights [0.4011, 0.1978, 0.4011] → output [3.0000, 4.0000]; rows 2 and 3 give [3.4067, 4.4067] and [3.5105, 4.5105]. Hand and code agree to four decimals.

Table 5. Attention unit tests (tolerance 1e-5).

Test	Result
Rows of attention weights sum to 1	max error 1.8×10^{-7}
Output vs F.scaled_dot_product_attention	within 2.4×10^{-7}
Causal mask	exactly 0 weight on future positions
Multi-head attention vs nn.MultiheadAttention	within 9×10^{-8}
Reshape (B, T, d) = (3, 10, 32) to (B, H, T, d/H)	(3, 4, 10, 8)

2.2 Multi-head attention and encoder block

W_Q, W_K, W_V, W_O are d × d; d is split into H heads of size d/H. The block is **pre-LayerNorm** — $x \leftarrow x + \text{Dropout}(\text{MHA}(\text{LN}_1(x)))$, $x \leftarrow x + \text{Dropout}(\text{FFN}(\text{LN}_2(x)))$, FFN = Linear(d, 4d) → ReLU → Linear(4d, d) — with sinusoidal positional encoding. Weights: $4d^2 + 2 \cdot d \cdot d_ff$ = **196,608** (d = 128, d_ff = 512); plus biases (1,152) and two LayerNorms (512) = **198,272** = sum(p.numel()).

2.3 Task A: encoder classifier on HAR

- Patches of 8 steps → linear embedding (d = 64) + CLS token → 2 encoder blocks, 4 heads → classifier on CLS; same split and budget as Problem 1.
- All three encoders beat the GRU (0.900); **no positional information scores highest (0.926 ± 0.011)**, at about 7× the parameters and 2.4× the time per epoch (Table 6).

Table 6. Task A encoders vs the best Problem 1 model (3 seeds each).

Model	Test accuracy	Macro-F1	Parameters	s / epoch
Encoder, no positional information	**0.9261 ± 0.0114**	**0.9257 ± 0.0112**	105,222	0.77
Encoder, learned positions	0.9140 ± 0.0110	0.9132 ± 0.0108	106,310	0.76
Encoder, sinusoidal	0.9034 ± 0.0098	0.9033 ± 0.0104	105,222	0.76
Best Problem 1 model (GRU, h = 64)	0.9001 ± 0.0066	0.8998 ± 0.0066	14,790	0.32

Interpretation: HAR classes differ mainly in the overall level and pattern of the signal in a window, not in the order of the 16 patches, so attention over an unordered set of patches is enough. Position information only helps if order matters; here it looks mildly distracting: accuracy falls from 0.926 (none) to 0.914 (learned) to 0.903 (sinusoidal) in Table 6. None vs sinusoidal (0.023) is about two standard deviations, none vs learned (0.012) about one, so the ordering is suggestive, not proven. The best encoder beats the GRU by 2.6 points for about 7× the parameters and 2.4× the time, so the GRU gives more accuracy per parameter.

2.4 Task B: character-level language model

- Context 128, d = 128, 4 heads, 4 layers, dropout 0.1 (810,049 parameters); AdamW, peak lr 1e-3, batch 64, 5,000 iterations.
- Validation loss **1.570 nats = 2.265 bits per character** (uniform 4.17 nats); training loss 1.357 (Fig. 6).
- Samples (500 characters, top-k = 20) at three temperatures are scored in Table 7 (real words, repeated 4-grams).

Table 7. Task B samples by sampling temperature.

Temperature	Words found in training text	Repeated 4-character sequences
0.2	98%	49%
0.8	95%	11%
1.5	74%	4%

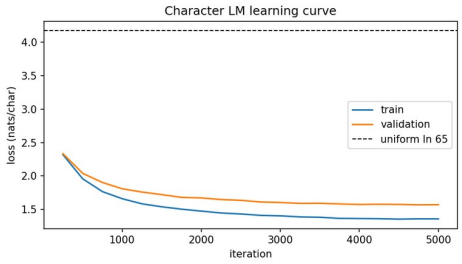


Figure 6. Training and validation loss of the character-level Transformer.

Interpretation: Temperature sets how close sampling stays to the most likely character (Table 7). At 0.2 almost every word is real (98%) but 49% of 4-character sequences repeat, so the text loops on common phrases. At 0.8 spelling is still good (95%) and repeats drop to 11%, giving varied, plausible text. At 1.5 the distribution is flat: real words fall to 74%, repeats to 4%, and invented words appear. Temperature trades correctness against diversity; 0.8 is the best balance here.

2.5 Analysis and ablations

(a) Heatmaps. The heads of the last layer look back only a few characters and put much of their weight on the previous character and on spaces (Table 8, Figure 7).

Table 8. Attention statistics of the four heads in layer 4.

Head	Average look-back (characters)	Weight on previous character	Weight on spaces
1	5.0	20%	10%
2	5.4	28%	23%
3	6.4	19%	7%
4	4.4	31%	28%

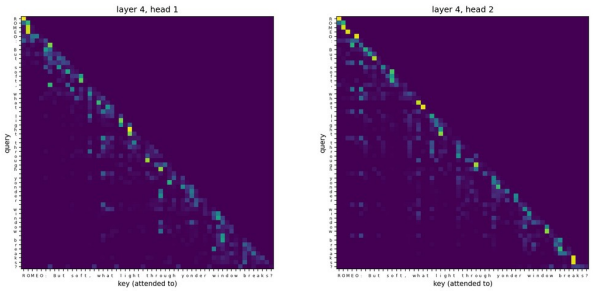


Figure 7. Attention weights of heads 1 and 2 (layer 4) on “ROMEO: But soft, what light through yonder window breaks?”.

Interpretation: Both heads are local: in Figure 7 most weight sits in a narrow band just below the diagonal. Head 2 is sharper, with a bright diagonal on the previous one or two characters (28% on the previous character, 23% on spaces, Table 8), so it acts as a previous-character and word-boundary head. Head 1 spreads weight over several earlier letters of the current word (20% previous character, 10% spaces), which fits a “word so far” head. Together they finish a word and mark where a new one starts.

(b) Heads and (c) causal mask. With 1,500 iterations each, the head count changes validation loss by at most 0.016 nats, while without the mask the training loss falls to **0.020 nats** but the sample is **299 newlines out of 300 characters** (Table 9, Figure 8).

Table 9. Ablations after 1,500 iterations.

Setting	Measured	Result
1 head	validation loss	1.810 nats
4 heads	validation loss	1.810 nats
8 heads	validation loss	1.826 nats
4 heads, causal mask	training loss	1.659 nats
4 heads, no mask	training loss	0.020 nats (sample: 299/300 newlines)

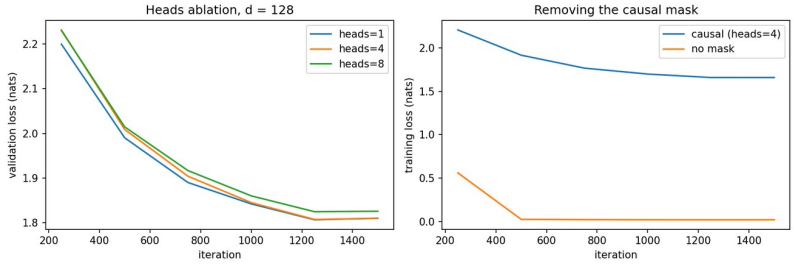


Figure 8. (a) Validation loss for 1, 4 and 8 heads. (b) Training loss with and without the causal mask.

Interpretation: The head count barely matters (Table 9): heads only re-partition the same d = 128 dimensions, and in 1,500 iterations a small character-level model learns mostly local patterns, which one head can represent. Without the causal mask, position t can attend to t+1, exactly the character it must predict, so the model copies it and the training loss collapses to 0.020. At generation time the next

character does not exist, so the shortcut fails and the sample is 299 newlines in 300 characters: low training loss without the mask measures leakage, not language modelling.

(d) Context length 64 → 512 (batch 16). Full model: 12.7 → 64.7 ms per step, 33 → 606 MB of activations. My attention alone: 1.2 → 8.8 ms, 4.5 → 260 MB, growing **3.8–3.9× per doubling** in memory — matching the $B{\cdot}H{\cdot}n^2$ weight matrices (Figure 9).

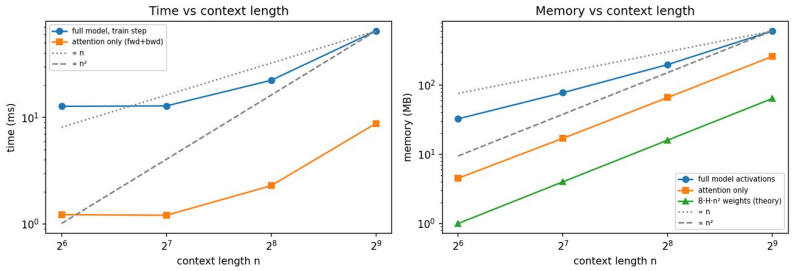


Figure 9. Time (left) and memory (right) against context length, with $\propto n$ and $\propto n^2$ reference lines.

Interpretation: Per layer the cost is $O(n^2{\cdot}d)$ for attention plus $O(n{\cdot}d^2)$ for the projections and feed-forward network. With d = 128 the linear terms dominate until n is several hundred (crossover near $6d \approx 770$), so in Figure 9 the full model's step time is flat from 64 to 128 and grows only 5× for 8× the context. The attention-only curves show the n^2 term: memory grows 3.8–3.9× per doubling (log-log slope 1.92–1.98), about 4× the $B{\cdot}H{\cdot}n^2$ theory line because several n^2 tensors are kept for the backward pass. Attention is 14% of the full model's memory at n = 64 but 43% at n = 512.

Bonus — LSTM vs Transformer. Table 10 and Figure 10 compare a 2-layer LSTM with the Transformer (5,000 iterations, 3 seeds). The LSTM wins on all three measures: lower validation loss in every seed (1.542 vs 1.563 nats, 2.225 vs 2.255 bits per character), a quarter of the training time and a third of the parameters.

Table 10. LSTM vs Transformer on the character-level task (mean ± std over 3 seeds).

Model	Parameters	Loss (nats)	Time per run (s)
2-layer LSTM	280,897	1.542 ± 0.007	45
Transformer	810,049	1.563 ± 0.006	198

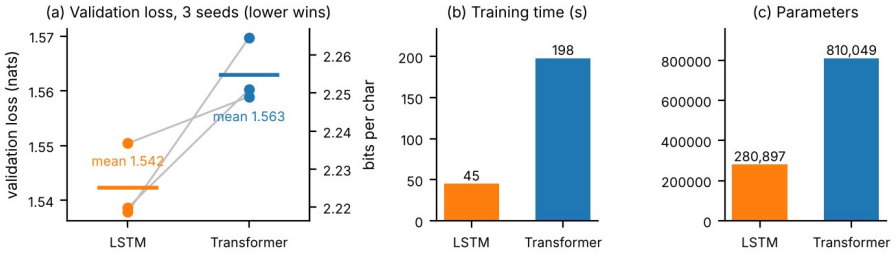


Figure 10. LSTM vs Transformer on the character task: (a) validation loss for each seed (grey lines join the same seed; right axis in bits per character), (b) training time, (c) parameters.

Interpretation: The LSTM is the better model here (Figure 10): most of the useful information in this small character-level task is local, which the LSTM's recurrent state captures by design, while the Transformer, with 2.9× the parameters, must learn locality from data (training-to-validation gap 0.21 nats). The LSTM's 0.021-nat lead is about three seed standard deviations and it trains 4.4× faster; the Transformer's strengths (parallelism, long-range lookup) go unused at context 128.

AI-use statement

Written by me: the from-scratch code (MyRNNCell, MyLSTMCell, MyGRUCell, scaled dot-product attention, sinusoidal positional encoding, multi-head attention and the encoder block); the hand derivations (parameter counts and the 3-token example); and the analysis and interpretation throughout, including the first version of the Problem 1 analysis and the Problem 2 interpretation paragraphs (sections 2.3, 2.4, 2.5 and the bonus).

Tool: Claude (Anthropic). Fixes and suggestions: reviewing my notebooks against the brief and suggesting fixes (changing the encoder block to pre-LayerNorm as specified; fixing a memory-measurement bug in the 5(d) cell; correcting factual errors in my Problem 1 analysis, T = 300 to 400, and updating numbers to the final run); checking my parameter counts and hand example against its own calculation; explaining the cell, attention and parameter-count equations; and debugging (a float32 precision issue in the split, Colab path and run problems).

Generated by Claude (supporting code and formatting, not the analysis): the test harnesses that compare my code with the PyTorch reference layers, the multi-seed LSTM-vs-Transformer comparison cell, README, requirements.txt and the zip-check script; and report assembly - section structure, layout, tables and figures with captions, the setup and method descriptions, summary tables, and condensing and extending my Problem 1 analysis (section 1.6) with figures taken from the tables. Claude also gave first drafts of the Problem 2 interpretation paragraphs, which I rewrote in my own words and checked against the results.

References

[1] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Computation, 1997.

[2] K. Cho et al., “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” EMNLP, 2014.

[3] A. Vaswani et al., “Attention is all you need,” NeurIPS, 2017.

[4] D. Anguita et al., “A public domain dataset for human activity recognition using smartphones,” ESANN, 2013.

[5] A. Karpathy, char-rnn: Tiny Shakespeare dataset, GitHub, 2015.

[6] PyTorch documentation, https://pytorch.org/docs.